

RaoRocketSim: Technical Report

Rao Bell Nozzle Design Toolbox — Architecture, Mathematics, and Usage

Auto-generated Repository Documentation

April 2026

Contents

1	Executive Summary	3
2	Repository Structure	3
2.1	Package Modules	3
3	Mathematical Foundations	4
3.1	Isentropic Gas Dynamics	4
3.2	Nozzle Contour Generation	4
3.2.1	Upstream Convergent Arc	4
3.2.2	Downstream Throat Arc	5
3.2.3	Divergent Bell Section	5
3.3	Method of Characteristics	5
3.4	Rao Variational Optimisation	6
3.5	Engine Performance	6
3.6	Flow Separation Criteria	6
3.7	Atmosphere Model	6
4	CLI Interface Walkthrough	6
4.1	Three Operating Modes	6
4.2	Key CLI Flags	7
4.3	Interactive Mode Flow	7
4.4	Build System	7
5	Propellant Database	8
6	Export Capabilities	8
6.1	CSV Export	8
6.2	STL Export	8
7	Analysis Features	8
7.1	Wall Pressure Distribution	8
7.2	Flow Separation Prediction	8
7.3	Altitude Performance Map	8
7.4	Combustion Chamber Sizing	9
7.5	Nozzle Comparison	9
7.6	Trade Studies	9
8	Capabilities Summary	9

9	Limitations	9
10	References	10

1 Executive Summary

RaoRocketSim is a Python-based engineering toolbox for designing thrust-optimised parabolic (TOP) bell rocket nozzles using the method originally developed by G. V. R. Rao (1958). The software computes nozzle contours via three distinct methods—Bézier approximation, Method of Characteristics (MoC) with numerical optimisation, and a calculus-of-variations (Rao variational) solver—and provides quasi-1D engine performance analysis, flow separation prediction, altitude performance mapping, combustion chamber sizing, trade studies, and CAD-ready export (CSV and STL).

The toolbox operates through a unified CLI (`main.py`) supporting interactive, batch, and parameter-sweep modes. It is implemented as a modular Python package (`raosim/`) containing 21 source modules, with 9 test files providing automated verification.

2 Repository Structure

Table 1: Top-level repository layout

Path	Description
<code>main.py</code>	CLI entry point (interactive/batch/sweep)
<code>raosim/</code>	Core Python package (21 modules)
<code>tests/</code>	Pytest test suite (9 files)
<code>builds/</code>	Versioned output artefacts (v001–v010)
<code>requirements.txt</code>	Dependencies: numpy, scipy, matplotlib
<code>Rocket_nozzle_sim_phase2.py</code>	Legacy Phase-2 prototype script

2.1 Package Modules

Table 2: `raosim/` module inventory

Module	Purpose
<code>gas_dynamics.py</code>	Isentropic relations, area–Mach, Prandtl–Meyer, c^* , C_f
<code>propellants.py</code>	Propellant database (4 built-in + custom)
<code>nozzle_geometry.py</code>	Bézier bell contour with Rao/NASA angle lookup tables
<code>moc.py</code>	Axisymmetric Method of Characteristics solver
<code>rao_optimizer.py</code>	MoC-coupled wall shape optimiser (Layer 2)
<code>rao_variational.py</code>	True Rao calculus-of-variations optimiser
<code>wall_model.py</code>	Cubic Hermite spline wall (Fritsch–Carlson monotone)
<code>engine.py</code>	Engine performance calculator (F , I_{sp} , $\dot{m}$, C_f)
<code>conical.py</code>	Conical (15°) nozzle baseline
<code>export.py</code>	CSV and binary STL export (with solid-body mode)
<code>plotting.py</code>	2D/3D nozzle visualisation and curvature plots
<code>wall_pressure.py</code>	Wall pressure $p(x)$ distribution and monotonicity check
<code>separation.py</code>	Flow separation (Summerfield, Kalt–Badal, Schmucker)
<code>chamber_geometry.py</code>	Combustion chamber + convergent section sizing
<code>altitude_performance.py</code>	Altitude sweep with separation tracking
<code>trade_study.py</code>	Parameter sweeps (ε , P_c , R_t) with plotting
<code>atmosphere.py</code>	Simplified ISA model (0–86 km)
<code>trajectory.py</code>	1D vertical-ascent trajectory integrator
<code>nozzle_comparison.py</code>	Multi-contour comparison with divergence loss
<code>build_log.py</code>	Versioned build directory and metadata logging

3 Mathematical Foundations

3.1 Isentropic Gas Dynamics

All flow relations assume a calorically perfect ideal gas with constant ratio of specific heats γ . The fundamental isentropic relations implemented in `gas_dynamics.py` are:

Temperature ratio:

$$\frac{T}{T_0} = \left(1 + \frac{\gamma - 1}{2} M^2\right)^{-1} \quad (1)$$

Pressure ratio:

$$\frac{p}{p_0} = \left(\frac{T}{T_0}\right)^{\gamma/(\gamma-1)} \quad (2)$$

Density ratio:

$$\frac{\rho}{\rho_0} = \left(\frac{T}{T_0}\right)^{1/(\gamma-1)} \quad (3)$$

Area–Mach relation:

$$\frac{A}{A^*} = \frac{1}{M} \left[\frac{2}{\gamma + 1} \left(1 + \frac{\gamma - 1}{2} M^2\right) \right]^{(\gamma+1)/[2(\gamma-1)]} \quad (4)$$

The supersonic root of (4) is inverted via Newton–Raphson with analytical derivatives.

Characteristic velocity:

$$c^* = \frac{\sqrt{\gamma R T_c}}{\gamma \sqrt{\left(\frac{2}{\gamma+1}\right)^{(\gamma+1)/(\gamma-1)}}} \quad (5)$$

Thrust coefficient:

$$C_f = \sqrt{\frac{2\gamma^2}{\gamma-1} \left(\frac{2}{\gamma+1}\right)^{(\gamma+1)/(\gamma-1)} \left[1 - \left(\frac{p_e}{p_c}\right)^{(\gamma-1)/\gamma}\right]} + \left(\frac{p_e}{p_c} - \frac{p_a}{p_c}\right) \varepsilon \quad (6)$$

Prandtl–Meyer function:

$$\nu(M) = \sqrt{\frac{\gamma+1}{\gamma-1}} \arctan \sqrt{\frac{\gamma-1}{\gamma+1} (M^2 - 1)} - \arctan \sqrt{M^2 - 1} \quad (7)$$

Inversion $M(\nu)$ uses Newton–Raphson with the analytical derivative $d\nu/dM = \sqrt{M^2 - 1}/[M(1 + \frac{\gamma-1}{2} M^2)]$.

3.2 Nozzle Contour Generation

The nozzle contour comprises three sections:

3.2.1 Upstream Convergent Arc

A circular arc of radius $R_u = 1.5 R_t$, centred at $(0, R_t + R_u)$, swept from angle $-(\pi/2 + \alpha_{\text{conv}})$ to $-\pi/2$, where $\alpha_{\text{conv}} = 45^\circ$ is the convergent half-angle.

3.2.2 Downstream Throat Arc

A circular arc of radius $R_d = 0.382 R_t$, centred at $(0, R_t + R_d)$, swept from $-\pi/2$ to $\theta_n - \pi/2$, producing the inflection point $N = (N_x, N_y)$.

3.2.3 Divergent Bell Section

Three methods are available:

Method 1: Quadratic Bézier (default). The bell from inflection N to exit $E = (L_n, R_e)$ is a quadratic Bézier curve with control point P_1 at the intersection of the tangent lines:

$$y = \tan \theta_n (x - N_x) + N_y \quad (\text{from } N) \quad (8)$$

$$y = \tan \theta_e (x - E_x) + E_y \quad (\text{from } E) \quad (9)$$

The parametric curve is:

$$\mathbf{B}(t) = (1-t)^2 \mathbf{N} + 2(1-t)t \mathbf{P}_1 + t^2 \mathbf{E}, \quad t \in [0, 1] \quad (10)$$

The angles θ_n and θ_e are obtained from Rao/NASA SP-8120 lookup tables via bilinear interpolation over $(\varepsilon, L\%)$.

The bell length is:

$$L_n = \frac{L\%}{100} \cdot L_{15^\circ} \quad \text{where} \quad L_{15^\circ} = \frac{R_e - R_t}{\tan 15^\circ} \quad (11)$$

Method 2: MoC + Numerical Optimisation. Uses coupled Method of Characteristics marching with SLSQP (or Nelder–Mead fallback) to find the wall shape maximising exit-plane thrust. Decision variables are $[\theta_n, c_1, \dots, c_n]$ (initial angle + spline control radii).

Method 3: Rao Variational. Implements Rao’s 1958 calculus-of-variations formulation: optimise the flow distribution (M, θ, ϕ) on a control surface CE by enforcing Euler–Lagrange stationarity, then construct the wall via design-mode MoC.

3.3 Method of Characteristics

The axisymmetric MoC solver (`moc.py`) implements the compatibility equations along C^+ and C^- characteristics (Anderson, Ch. 11):

$$C^+ : \quad d\theta + d\nu = Q^+ ds, \quad Q^+ = \frac{\sin \theta \sin \mu \cos \mu}{r \cos(\theta + \mu)} \quad (12)$$

$$C^- : \quad d\theta - d\nu = Q^- ds, \quad Q^- = -\frac{\sin \theta \sin \mu \cos \mu}{r \cos(\theta - \mu)} \quad (13)$$

Three unit processes are implemented:

1. **Interior point:** C^- from upper parent $\cap$ C^+ from lower parent, predictor-corrector with source terms.
2. **Axis point:** symmetry BC ($\theta = 0, r = 0$), with $\sin \theta/r$ singularity handling.
3. **Wall point:** tangency BC ($\theta_{\text{flow}} = \theta_{\text{wall}}$), iterative C^+ intersection.

The starting line is approximated via either quasi-1D area–ratio mapping or a Hall-style transonic correction with coefficients $a_1 = \sqrt{2/[(\gamma+1)\rho_c]}$ and $a_2 = (\gamma+1)/(12\rho_c)$ where $\rho_c = R_d/R_t$.

3.4 Rao Variational Optimisation

The augmented functional over the control surface CE is:

$$I = \int_{CE} (f_1 + \lambda_2 f_2 + \lambda_3 f_3) dr \quad (14)$$

where f_1 is the thrust integrand, f_2 the mass-flow integrand, and $f_3 = \cot \phi$ the length integrand. Stationarity requires:

$$\frac{\partial}{\partial M}(f_1 + \lambda_2 f_2) = 0, \quad \frac{\partial}{\partial \theta}(f_1 + \lambda_2 f_2) = 0, \quad \frac{\partial}{\partial \phi}(f_1 + \lambda_2 f_2 + \lambda_3 f_3) = 0 \quad (15)$$

These are solved via Newton iteration at each CE station with numerical Jacobians, embedded in an outer loop updating λ_2 , λ_3 from constraint residuals. Transversality at the exit endpoint is also enforced.

3.5 Engine Performance

From `engine.py`, the key performance parameters are:

$$M_e = M\left(\frac{A_e}{A_t}, \gamma\right) \quad (\text{from area-Mach}) \quad (16)$$

$$C_{f,\text{actual}} = C_{f,\text{ideal}} \cdot \eta_{I_{sp}} \quad (17)$$

$$F = C_{f,\text{actual}} \cdot P_c \cdot A_t \quad (18)$$

$$\dot{m} = \frac{P_c A_t}{c^*} \quad (19)$$

$$I_{sp} = \frac{C_{f,\text{actual}} \cdot c^*}{g_0} \quad (20)$$

3.6 Flow Separation Criteria

Three empirical criteria are implemented:

$$\text{Summerfield: } p_{\text{sep}} \approx 0.4 p_a \quad (21)$$

$$\text{Kalt-Badal: } \frac{p_{\text{sep}}}{p_a} \approx \frac{1}{1.88 M_e - 1} \quad (22)$$

$$\text{Schmucker: } \frac{p_{\text{sep}}}{P_c} \approx \frac{(p_a/P_c)^{0.8}}{M_e} \quad (23)$$

3.7 Atmosphere Model

A simplified 3-segment ISA: troposphere (lapse -6.5 K/km to 11 km), tropopause (isothermal 216.65 K to 20 km), stratosphere (lapse $+1.0$ K/km to 47 km), and exponential tail above.

4 CLI Interface Walkthrough

4.1 Three Operating Modes

The CLI (`main.py`) supports:

1. **Interactive mode:** `python main.py` — guided prompts for all parameters.
2. **Batch mode:** all parameters via flags, e.g.:

```

1 python main.py --propellant LOX/RP-1 --Pc 45 --Rt 20 \
2   --epsilon 10 --method moc --stl nozzle.stl

```

3. Sweep mode: parametric trade studies:

```

1 python main.py --propellant LOX/LCH4 --Pc 60 --Rt 25 \
2   --sweep epsilon 4 50 20

```

4.2 Key CLI Flags

Table 3: Principal CLI arguments

Flag	Default	Description
--propellant	—	Propellant name (LOX/RP-1, LOX/LCH4, LOX/LH2, N2O/Ethanol)
--Pc	—	Chamber pressure [bar]
--Pa	101.325	Ambient pressure [kPa]
--Rt	—	Throat radius [mm]
--epsilon	—	Expansion ratio A_e/A_t
--method	bezier	Contour method: bezier , moc , or rao
--length-pct	80	Bell length % of 15° cone
--compare	off	Side-by-side contour comparison
--wall-pressure	off	Wall pressure distribution
--separation	off	Flow separation check
--altitude-map	off	Altitude performance sweep
--chamber	off	Combustion chamber geometry
--output	—	CSV output path
--stl	—	STL output path
--sweep	—	Parameter sweep (VAR MIN MAX N)
--gamma/--Mw/--Tc	—	Custom propellant properties

4.3 Interactive Mode Flow

1. Select propellant from database or enter custom (γ, M_w, T_c).
2. Enter chamber pressure P_c , ambient pressure P_a , throat radius R_t .
3. Choose expansion ratio mode: compute from P_c/P_a , specify ε , or specify R_e .
4. Select bell length percentage and contour method.
5. View engine performance summary (thrust, I_{sp} , $\dot{m}$, C_f , c^* , M_e , P_e).
6. Run separation check (automatic).
7. Optional: wall pressure distribution, chamber geometry, altitude map.
8. Export CSV and STL to versioned **builds/** directory.
9. View 2D/3D plots.

4.4 Build System

Every run creates a versioned directory **builds/vNNN_YYYYMMDD_HHMMSS/** containing all output files and a **metadata.txt** log with full input parameters and performance results.

5 Propellant Database

Table 4: Built-in propellant combinations

Propellant	γ	M_w [g/mol]	T_c [K]	$\eta_{I_{sp}}$	O/F
N2O/Ethanol	1.22	26.0	2800	0.92	5.5
LOX/RP-1	1.23	23.5	3400	0.96	2.6
LOX/LCH4	1.24	22.0	3500	0.96	3.5
LOX/LH2	1.20	10.0	3250	0.98	6.0

Custom propellants can be specified via `--gamma`, `--Mw`, and `--Tc` flags.

6 Export Capabilities

6.1 CSV Export

Outputs (x, y) contour coordinates in metres with configurable point count (default 301). Format: two-column CSV with header `x_m,y_m`.

6.2 STL Export

Revolves the 2D contour around the axis to produce a 3D surface of revolution as a binary STL file. Two modes:

- **Inner surface only:** reference geometry for CAD/CAM.
- **Solid body:** closed manifold with inner surface, offset outer wall, sealed annular end-caps, and optional inlet flange—directly 3D-printable or CNC-ready.

The offset contour is computed by displacing each point along its surface normal by the specified wall thickness. Monotone Fritsch–Carlson interpolation prevents self-intersection.

7 Analysis Features

7.1 Wall Pressure Distribution

Uses local area ratio $A(x) = \pi y(x)^2$ to compute $M(x)$ via the area–Mach relation (subsonic upstream of throat, supersonic downstream), then $p(x) = P_c \cdot (p/p_0)(M)$. A monotonicity check flags regions where $dp/dx > 0$ downstream of the throat as separation risk.

7.2 Flow Separation Prediction

The separation pressure p_{sep} is compared against the local wall pressure. If $p_e < p_{\text{sep}}$, separation is predicted and the axial location is identified by marching downstream until $p(x) \leq p_{\text{sep}}$.

7.3 Altitude Performance Map

Sweeps altitude 0–100 km, computing thrust, I_{sp} , and C_f at each altitude using the ISA pressure profile. Separation status is tracked; the altitude at which separation clears is reported.

7.4 Combustion Chamber Sizing

Parameterised by characteristic length L^* and contraction ratio A_c/A_t . The cylindrical length is computed from the required chamber volume minus the convergent frustum volume:

$$L_c = \frac{V_{\text{chamber}} - V_{\text{frustum}}}{A_c}, \quad V_{\text{chamber}} = L^* \cdot A_t \quad (24)$$

7.5 Nozzle Comparison

Generates conical, Bézier, and Rao contours side-by-side with quantitative metrics: divergence loss η_{div} , corrected C_f , nozzle efficiency η_n , and geometric parameters. The divergence loss is estimated by integrating axial momentum across the exit plane with a linear flow-angle profile assumption.

7.6 Trade Studies

Parametric sweeps over ε , P_c , or R_t with multi-panel plots of I_{sp} , C_f , thrust, nozzle length, and mass flow rate. Results are exported as CSV in the build directory.

8 Capabilities Summary

- Three contour generation methods (Bézier, MoC-optimised, Rao variational)
- Four built-in bipropellant combinations + custom propellant support
- Complete quasi-1D engine performance (F , I_{sp} , $\dot{m}$, C_f , c^*)
- Three flow separation criteria (Summerfield, Kalt–Badal, Schmucker)
- Wall pressure distribution with monotonicity validation
- Altitude performance mapping (0–100 km) with separation tracking
- Combustion chamber geometry sizing (L^* , contraction ratio)
- Conical vs. bell nozzle comparison with divergence loss quantification
- Parametric trade studies (ε , P_c , R_t sweeps)
- CSV and STL export (inner surface or solid body with optional flange)
- 2D/3D visualisation with curvature plotting
- Versioned build system with metadata logging
- 1D vertical-ascent trajectory integrator
- ISA atmosphere model (0–86 km)
- Interactive, batch, and sweep CLI modes
- Automated test suite (9 test files)

9 Limitations

1. **Calorically perfect gas assumption:** constant γ throughout. Real exhaust gases exhibit temperature-dependent γ and dissociation/recombination; frozen-flow or equilibrium chemistry is not modelled.
2. **Quasi-1D performance:** engine performance uses 1D isentropic relations. Boundary-layer losses, heat transfer, and viscous effects are captured only through the empirical $\eta_{I_{sp}}$ correction factor.
3. **Starting-line approximation:** the transonic starting line for MoC uses either a quasi-1D area-ratio estimate or a simplified Hall polynomial—neither is a full transonic solution (Hall 1962 / Kliegel–Levine 1969).

4. **No boundary-layer modelling:** wall friction, displacement thickness, and heat transfer are not computed. The contour represents the inviscid flowpath.
5. **Simplified separation criteria:** the three empirical correlations (Summerfield, Kalt-Badal, Schmucker) are engineering approximations; no CFD-based or boundary-layer-integral separation model is included.
6. **No two-phase flow:** condensation, particulate drag, and droplet breakup in the nozzle are not modelled (relevant for solid and some hybrid motors).
7. **Steady-state only:** transient start-up, shutdown, and throttling dynamics are not considered.
8. **Axisymmetric geometry only:** 3D effects (e.g., from non-axisymmetric inlets, film cooling ports, or scarfed exits) cannot be captured.
9. **No thermal/structural analysis:** wall temperatures, thermal stresses, and structural margins are outside scope.
10. **Linear exit-flow assumption in divergence loss:** the nozzle comparison module assumes a linear flow-angle distribution from axis to wall at the exit plane, which is approximate for shaped bells.
11. **Propellant properties are tabulated, not computed:** the γ , M_w , T_c values are representative averages, not computed from chemical equilibrium at the actual O/F and P_c .
12. **Trajectory module is rudimentary:** the 1D vertical-ascent integrator uses constant thrust, constant I_{sp} , and a simple drag model—it is suitable for rough estimates, not mission design.

10 References

1. G. V. R. Rao, “Exhaust Nozzle Contour for Optimum Thrust,” *Jet Propulsion*, 1958.
2. G. V. R. Rao, “Recent Developments in Rocket Nozzle Configurations,” *ARS Journal*, 1961.
3. NASA SP-8120, “Liquid Rocket Engine Nozzles,” 1976.
4. J. D. Anderson, *Modern Compressible Flow*, 3rd ed., McGraw-Hill, 2003.
5. M. J. Zucrow and J. D. Hoffman, *Gas Dynamics*, Vol. 2, Wiley, 1977.
6. I. M. Hall, “Transonic Flow in Two-Dimensional and Axially-Symmetric Nozzles,” *QJ-MAM*, 15(4), 1962.
7. J. R. Kliegel and V. Levine, “Transonic Flow in Small Throat Radius Nozzles,” *AIAA J.*, 7(7), 1969.
8. R. Stark, “Flow Separation in Rocket Nozzles — An Overview,” 2009.